AGENTIC RAG

Arquitetura e Roadmap de Implementação

Especificação técnica para desenvolvimento incremental com Codex

Versão 1.0

17 de agosto de 2026

> **Objetivo do documento**
> Servir como fonte de verdade para o Codex. A arquitetura-alvo fica definida aqui, mas cada etapa deve ser implementada, testada e aceita antes da próxima. O foco inicial e provar qualidade de ingestão e retrieval antes de adicionar comportamento agentico, fallback web ou hardening de produção.

1. Objetivo e escopo

Construir uma API de chat baseada em LangGraph que responda perguntas utilizando primeiro uma base de conhecimento interna em Chroma. Quando o contexto interno for insuficiente, o fluxo reescreve a consulta e tenta novamente; persistindo a insuficiência, usa busca web como fallback. A resposta final deve ser transmitida ao frontend por streaming e acompanhar suas fontes.

> **Decisão arquitetural central**
> LangGraph orquestra o processo; ele não deve conter implementacoes de Chroma, reranking, web search, provider de LLM ou transporte HTTP. Esses elementos ficam desacoplados atras de componentes proprios.

Escopo da v1

- API de chat com streaming.
- Pipeline de ingestão de documentos separado do fluxo de perguntas.
- Chroma como vector database.
- Retrieval em dois estágios: dense retrieval com diversidade por MMR e reranking com CrossEncoder.
- LangGraph com query preparation, retrieval, grading, query rewrite, fallback web, generation e no-answer.
- Memória conversacional por thread/conversationId.
- Citações e metadados das fontes utilizadas.
- Suite mínima de avaliação do retrieval e critérios de produção.

Fora do escopo inicial

- Multi-agent ou crews de especialistas.
- Autonomia irrestrita para o LLM escolher qualquer ferramenta.
- Hybrid retrieval lexical+dense no primeiro milestone; a arquitetura deve permitir a evolução posterior.

- Fine-tuning de embedding ou reranker antes de existir um dataset de avaliação.
- Infraestrutura de produção antes de o fluxo funcional estar comprovado.

2. Princípios arquiteturais

Principio	Aplicação pratica
Orquestração explícita	Decisões de fluxo importantes são edges/rotas do LangGraph, não comportamento implicito de um agente livre.
Duas pipelines independentes	Knowledge ingestion e query/chat são sistemas distintos que se encontram apenas no vector store/retriever.
Retrieval como componente	O grafo chama KnowledgeRetriever.retrieve(query); desconhece Chroma, MMR, modelos e parâmetros internos.
Providers substituiveis	LLM, Web Search, embeddings, vector store e checkpointer devem poder ser trocados com impacto local.
Streaming como contrato	Frontend consome um protocolo proprio e estavel; detalhes de streaming do LangGraph ficam atras de um adapter.
Progressao incremental	Nenhuma etapa futura deve ser implementada antecipadamente. Cada milestone precisa permanecer executavel e testavel.
Evidência antes de resposta	Ter resultados não significa ter contexto suficiente. RAG e web precisam de avaliação de relevância/suficiência.
Sem loops abertos	Tentativas de retrieval e rewrite possuem limites configuráveis.

3. Arquitetura planejada

A aplicação é organizada em camadas com responsabilidades claras. O frontend conversa apenas com a API. A API executa o grafo e traduz os eventos do runtime para o protocolo de streaming. O grafo consome serviços de retrieval, web search e LLM sem conhecer detalhes de infraestrutura.

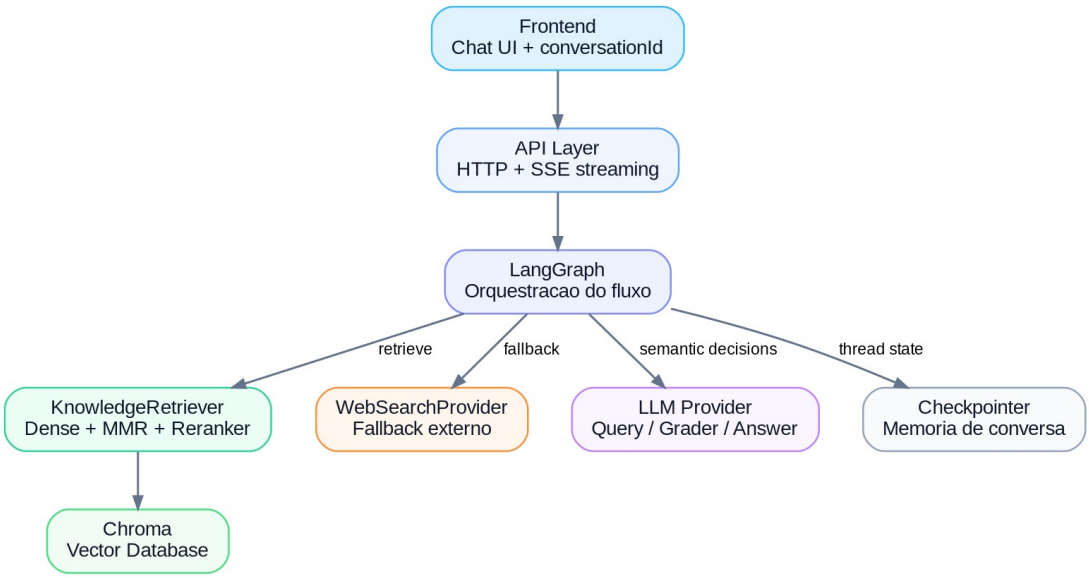


Figura 1 - Arquitetura de alto nível

Responsabilidades por camada

Componente	Responsabilidade
Frontend	Chat UI, conversationId, renderizacao progressiva da resposta, exibicao de status e fontes.
API Layer	Validação HTTP, autenticação/autorização futura, abertura do stream, tratamento de erros e cancelamento.
LangGraph	Estado, nodes, conditional edges, limites de tentativa e orquestração do workflow.
KnowledgeRetriever	Busca de candidatos, MMR, reranking e retorno de documentos ranqueados.
Chroma	Persistência de embeddings, texto e metadata dos chunks.
WebSearchProvider	Busca externa. Nenhum provider concreto deve vazar para o grafo.
LLM Provider	Modelos para preparar query, grading, rewrite e resposta final.
Checkpointer	Persistência do estado de threads/conversas; separado da base de conhecimento.

4. Pipeline de knowledge ingestion

A ingestão prepara e indexa conhecimento. Ela deve poder rodar fora do processo de chat e ser testada independentemente. O resultado final é uma collection no Chroma com chunks, embeddings e metadata consistente.

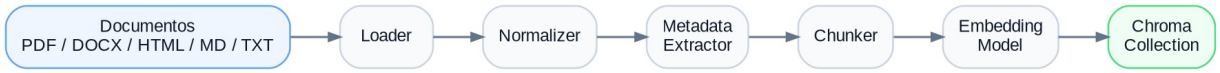


Figura 2 - Pipeline de ingestão

Loader

Converte PDF, DOCX, HTML, Markdown, TXT ou outras fontes em uma representação comum de documento.

Normalizer

Remove ruído previsível, normaliza espacos/quebras, identifica conteúdo vazio e preserva estrutura útil.

Metadata extractor

Anexa identificadores e informações necessárias para filtros, auditoria e citações.

Chunker

Divide o conteúdo em unidades recuperaveis. O algoritmo deve ficar encapsulado para permitir evolução futura.

Embedding model

Gera vetores dos chunks e deve ser configurável.

Chroma

Armazena identificador, documento/chunk, embedding e metadata. Filtros por metadata devem ser possíveis.

> **Metadata mínima por chunk**
> document_id, title, source, section, page (quando aplicável), chunk_index, created_at, updated_at e url (quando aplicável). IDs de chunks devem ser estáveis o suficiente para reindexação e rastreabilidade.

5. Arquitetura de retrieval

O retrieval inicial deve privilegiar qualidade sem transformar o primeiro milestone em um motor de busca completo. A estrategia aprovada para v1 e recuperar um conjunto relativamente amplo de candidatos com busca vetorial, aplicar MMR para reduzir redundância e usar CrossEncoder para reordenar apenas esse conjunto.

```
QUERY
  |
  v
Embedding da query
  |
  v
Chroma dense retrieval
  |
  v
MMR (relevância + diversidade)
  |
  v
~20 candidatos
  |
  v
CrossEncoder reranker
  |
  v
Top ~5 chunks
  |
  v
RAG grader
```

Contrato conceitual

```
KnowledgeRetriever.retrieve(query, filters?) -> RetrievalResult

RetrievalResult
  documents[]
    content
    metadata
    retrieval_score?      # quando disponivel
    rerank_score
  diagnostics             # opcional, não exposto ao usuário por padrão
```

Parâmetros como fetch_k, candidate_k, final_k e lambda_mult devem ser configuração. Valores iniciais são hipóteses de engenharia e precisam ser calibrados com um dataset real; não devem ser tratados como thresholds universais.

> **Evolução prevista**
>
> A interface KnowledgeRetriever deve permitir substituir Chroma + MMR por uma estrategia hybrid (dense + lexical/BM25 + Reciprocal Rank Fusion + reranker) sem alterar nodes ou edges do LangGraph.

6. LangGraph: estado, nodes e roteamento

O grafo implementa o processo de decisão. Nodes devem ter uma responsabilidade única e retornar apenas atualizacoes de estado. Integrações externas são chamadas por componentes injetados ou factories, nunca implementadas diretamente no grafo.

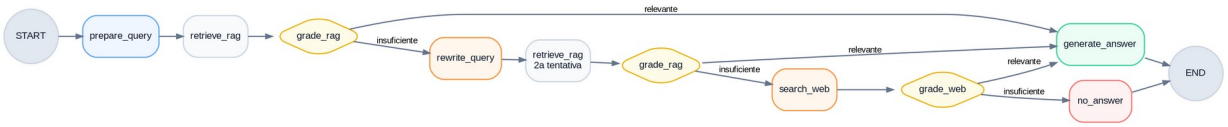


Figura 3 - Fluxo-alvo do Agentic RAG

Estado conceitual

```
AgentState
  messages
  question
  search_query
  documents
  rag_attempts
  rag_relevant
  rag_confidence
  web_results
  web_relevant
  source          # rag | web | none
  answer
```

Node	Responsabilidade
prepare_query	Usa a conversa apenas para resolver contexto e produz uma consulta independente e pesquisavel.
retrieve_rag	Chama KnowledgeRetriever e grava os documentos recuperados no state.
grade_rag	Avalia se o contexto é suficiente para responder; não gera a resposta.
rewrite_query	Reformula a consulta após retrieval insuficiente; não responde ao usuário.
search_web	Chama WebSearchProvider somente depois de esgotar as tentativas internas.
grade_web	Avalia se resultados externos fornecem evidência suficiente/confiável.
generate_answer	Único node responsável pela resposta final; recebe pergunta, contexto e fontes.
no_answer	Finaliza com resposta segura quando não existe evidência suficiente.

> **Política de loop**
> Comecar com no máximo 2 retrievals internos: tentativa original -> rewrite -> segunda tentativa. Se ainda for insuficiente, seguir para web. O limite deve ser configurável.

7. API e contrato de streaming com o frontend

A API deve expor um contrato de streaming proprio. O frontend não deve depender diretamente do formato interno de eventos do LangGraph. Isso permite trocar a estrategia de streaming do runtime sem alterar a UI.

Endpoint planejado

```
POST /chat/stream

request:
  conversationId: string
  message: string

response:
  text/event-stream (SSE)
```

Eventos públicos

Evento	Finalidade
status	Mudança de etapa visível ao usuário. Ex.: rag_search, reranking, query_rewrite, web_search, generating.
sources	Lista normalizada de fontes que poderão ser citadas/exibidas.
token	Fragmento incremental da resposta final.
done	Execução concluída normalmente.
error	Falha tratada; não deve vazar stack trace, prompt interno ou segredo.

```
event: status
data: {"stage":"rag_search","message":"Buscando na base de conhecimento..."}

event: sources
data: {"sources":[...]}

event: token
data: {"text":"Para"}

event: token
data: {"text":" configurar"}

event: done
data: {}
```

Regra de privacidade do stream

Tokens e saidas de grader, rewrite ou outros passos internos não devem ser transmitidos ao usuário. O stream de tokens deve ser filtrado para a geração final da resposta.

8. Conversação, memória e persistência

O identificador de conversa vindo do frontend será usado como thread_id do LangGraph. Isso permite que mensagens subsequentes compartilhem contexto e que prepare_query transforme perguntas dependentes do histórico em consultas independentes para retrieval.

```
Frontend conversationId = "87da97..."
                  |
                  v
LangGraph configurable.thread_id = "87da97..."
```

Ambiente	Checkpointer
Desenvolvimento	In-memory, apenas para facilitar execução local e testes.
Produção	Persistência durável (preferencialmente PostgreSQL ou backend aprovado pela plataforma).

Separacao de dados

Chroma persiste conhecimento indexado. O checkpointer persiste estado de execução/conversa. Não usar um para substituir a responsabilidade do outro.

9. Organização lógica do projeto

Os nomes exatos de arquivos podem variar; o Codex deve preservar os limites de responsabilidade abaixo. Evitar arquivos gigantes que misturam HTTP, grafo, banco vetorial e prompts.

```
app/
  api/              # HTTP, validação, SSE, auth futura
  graph/            # state, nodes, routes, graph builder
  knowledge/
    ingestion/      # loaders, normalization, chunking
    embeddings/     # provider/config de embeddings
    vectorstore/    # adapter Chroma
    retrieval/      # MMR, reranker, KnowledgeRetriever
  websearch/        # WebSearchProvider + implementação concreta
  llm/              # modelos/prompts/structured outputs
  streaming/        # LangGraph events -> protocolo do frontend
  config/           # settings e validação de ambiente
  main              # bootstrap da aplicação
```

Regra de dependências

graph depende de interfaces/serviços de domínio; knowledge, websearch e llm não dependem da API. A camada HTTP chama o grafo, nunca o contrario.

10. Roadmap incremental de implementação

A ordem abaixo é parte da especificação. O Codex deve trabalhar uma etapa por vez e declarar explicitamente o que foi criado, como executar e como verificar os critérios de aceite antes de iniciar a próxima.

Etapa 1 Bootstrap da API

Objetivo. Criar um projeto Python executavel, com configuração e uma API mínima.

Entregas

- Estrutura base do projeto.
- FastAPI (ou framework acordado), settings e carregamento de ambiente.
- Endpoint GET /health.
- Testes básicos e comandos de execução.

Critérios de aceite

- Aplicação inicia localmente sem dependências externas obrigatórias.
- GET /health retorna 200.
- Testes da etapa passam.

> **Não antecipar nesta etapa**
>
> Não adicionar LangGraph, Chroma, embeddings, web search ou streaming.

Etapa 2 Knowledge ingestion

Objetivo. Provar que documentos podem ser carregados, normalizados, fragmentados e indexados.

Entregas

- Modelo comum de Document/Chunk.
- Loader inicial para um conjunto pequeno de formatos prioritários.
- Normalizer, metadata extractor e Chunker encapsulados.
- EmbeddingProvider configurável.
- Adapter Chroma e comando/serviço de ingestão.

Critérios de aceite

- Um documento de teste gera chunks com metadata válida.
- Os chunks são persistidos no Chroma e podem ser inspecionados/contados.
- Reindexação não cria comportamento imprevisivel de IDs.

> **Não antecipar nesta etapa**
>
> Não criar grafo nem resposta por LLM. O objetivo é apenas construir uma base recuperavel.

Etapa 3 KnowledgeRetriever

Objetivo. Implementar e medir retrieval isoladamente antes de usar LangGraph.

Entregas

- KnowledgeRetriever como API única de retrieval.
- Dense retrieval no Chroma.
- MMR para diversidade.
- CrossEncoder para reranking.
- Configuração de fetch_k/candidate_k/final_k/lambda_mult.
- Testes unitarios e teste manual com perguntas conhecidas.

Critérios de aceite

- Perguntas de teste retornam chunks semanticamente relacionados.
- Reranking altera a ordem de candidatos quando apropriado.
- LangGraph ainda não é necessário para executar o retriever.

> **Não antecipar nesta etapa**
>
> Se a qualidade estiver ruim, corrigir ingestão/chunking/retrieval antes de seguir. Não usar LLM para mascarar retrieval ruim.

Etapa 4 RAG básico no LangGraph

Objetivo. Provar o caminho ponta a ponta mais simples.

Entregas

- State mínimo.
- Nodes prepare_query, retrieve_rag e generate_answer.
- Grafo START -> prepare -> retrieve -> answer -> END.

Critérios de aceite

- Uma pergunta usa documentos do Chroma para produzir resposta.
- A resposta final não utiliza contexto fora dos documentos recuperados quando o prompt exige grounding.
- Nodes continuam pequenos e testaveis.

> **Não antecipar nesta etapa**
>
> Não adicionar grader, rewrite, web fallback ou memória persistente.

Etapa 5 RAG Grader

Objetivo. Separar claramente a existência de resultados da suficiência do contexto.

Entregas

- Structured output para relevant/confidence.
- Node grade_rag.
- Rota relevant -> generate_answer; insufficient -> no_answer temporario.

Critérios de aceite

- Casos relevantes e irrelevantes são distinguíveis em testes.
- Grader não responde a pergunta.
- Saída é estruturada e validada.

> **Não antecipar nesta etapa**
>
> Não adicionar query rewrite ainda; primeiro comprovar que o grader funciona.

Etapa 6 Query rewrite e retry

Objetivo. Adicionar autocorreção controlada ao retrieval.

Entregas

- Node rewrite_query.
- Rota de retorno para retrieve_rag.
- Contador rag_attempts e limité configurável.

Critérios de aceite

- Fluxo faz no máximo o número permitido de tentativas.
- Consulta reescrita é observável em logs/traces, não no frontend.
- Não existe possibilidade de loop infinito.

> **Não antecipar nesta etapa**
>
> Não adicionar web search antes de fechar o retry interno.

Etapa 7 Streaming para o frontend

Objetivo. Transformar a execução em experiência de chat responsiva.

Entregas

- POST /chat/stream.
- Adapter de streaming.
- Eventos status, sources, token, done, error.
- Filtro para transmitir tokens apenas da resposta final.

Critérios de aceite

- Frontend/cliente de teste recebe status antes da resposta.
- Tokens chegam progressivamente.
- Eventos internos de grader/rewrite não vazam.
- Conexão encerra com done ou error.

> **Não antecipar nesta etapa**
>
> Não acoplar o frontend ao formato interno de eventos do LangGraph.

Etapa 8 Threads e memória conversacional

Objetivo. Permitir follow-ups coerentes sem degradar a consulta de retrieval.

Entregas

- Mapeamento conversationId -> thread_id.
- Checkpointer de desenvolvimento.
- prepare_query usando histórico somente para resolver referências/contexto.

Critérios de aceite

- Pergunta "E no Keycloak?" após uma pergunta sobre OAuth vira uma search_query independente.
- Duas conversations IDs diferentes não compartilham estado.

> **Não antecipar nesta etapa**
>
> Não confundir memória conversacional com base de conhecimento.

Etapa 9 Web fallback

Objetivo. Usar conhecimento externo somente depois de esgotar a base interna.

Entregas

- Interface WebSearchProvider.
- Implementação concreta configurável.
- Nodes search_web e grade_web.
- Rotas para generate_answer ou no_answer.

Critérios de aceite

- Web search não é chamada quando RAG é suficiente.
- Resultados web possuem metadata de fonte/URL.
- Contexto web insuficiente termina em no_answer, sem alucinação.

> **Não antecipar nesta etapa**
>
> Provider concreto não pode aparecer dentro de nodes fora do adapter/serviço apropriado.

Etapa 10 Citações e proveniência

Objetivo. Tornar a resposta auditavel e útil para o frontend.

Entregas

- Modelo normalizado de Source.
- IDs de citação [1], [2]...

- Mapeamento entre cada citação e chunk/URL.
- Evento sources e resposta final consistentes.

Critérios de aceite

- Toda citação exibida aponta para uma fonte existente.
- Metadados de página/seção/url são preservados quando disponiveis.
- Não inventar referências.

Etapa 11 Avaliação

Objetivo. Substituir ajustes por intuição por experimentação reproduzivel.

Entregas

- Dataset inicial com perguntas reais e documentos esperados.
- Métricas de retrieval: Recall@K e pelo menos uma métrica de ranking como MRR/nDCG.
- Avaliações de grounding/relevância de resposta.
- Script/relatorio comparavel entre configurações.

Critérios de aceite

- Mudanças em chunking, embedding, MMR ou reranker podem ser comparadas numericamente.
- Existe baseline versionado antes de otimizar.

> **Não antecipar nesta etapa**
>
> Não trocar modelos apenas por preferencia; registrar ganho, custo e latência.

Etapa 12 Hardening de produção

Objetivo. Adicionar confiabilidade e controles operacionais sem alterar o desenho de domínio.

Entregas

- Checkpointer durável.
- Timeouts, retries técnicos e circuit breaking onde fizer sentido.
- Observabilidade/tracing/métricas.
- Rate limiting, autenticação/autorização e políticas de acesso.
- Tratamento de cancelamento do stream.
- Docker/deploy e configuração por ambiente.

Critérios de aceite

- Nenhum segredo em código/logs.
- Falhas externas são degradadas de forma controlada.
- Threads sobrevivem a restart quando requerido.
- SLOs/métricas essenciais são observaveis.

11. Critérios de aceite da v1 completa

- Uma pergunta chega por API, recebe streaming de status e tokens e termina de forma determinística.
- O sistema usa primeiro a base Chroma; web só é chamada após contexto interno insuficiente e retry controlado.
- Resposta final é grounded nas fontes selecionadas e fornece citações rastreáveis.
- Retriever pode ser testado e substituido sem alterar o grafo.
- Web provider pode ser trocado sem alterar o grafo.
- conversationId isola threads e preserva contexto quando habilitado.
- Nenhum output interno de grading/rewrite vaza para o frontend.
- Existe dataset mínimo para avaliar retrieval antes de otimizar configurações.
- Configurações críticas são externalizadas; segredos não ficam no repositório.
- Testes e instruções de execução acompanham cada módulo relevante.

12. Instruções operacionais para o Codex

> **Regra principal**
> Implemente somente a etapa solicitada. Não crie código de etapas posteriores "para deixar pronto". Ao terminar, entregue resumo das mudanças, árvore de arquivos afetada, como executar, testes executados e eventuais decisões tomadas.

Regra	Orientação
Preservar limites	LangGraph = orquestração. KnowledgeRetriever = retrieval. Chroma = storage vetorial. API = transporte. StreamingAdapter = protocolo frontend.
Preferir interfaces claras	Não criar abstrações por abstração; criar interfaces quando existe um limite de substituicao planejado: retriever, LLM, web search, vector store/checkpointer quando aplicável.
Não esconder falhas	Se uma etapa não cumprir critério de aceite, corrigir nela antes de avancar.
Testabilidade	Funções de domínio e adapters devem poder ser testados sem subir a aplicação inteira quando razoável.
Configuração	Modelos, K values, timeouts, URLs e chaves ficam em settings/environment; evitar magic numbers espalhados.
Prompts	Prompts internos devem ter finalidade única e structured outputs quando a decisão alimentar roteamento.
Observabilidade sem vazamento	Logs podem registrar IDs, duração, contagens e scores; não registrar segredos nem chain-of-thought.
Mudanças pequenas	Cada etapa deve preferir commits/diffs pequenos e revisáveis, mantendo a aplicação executavel.

Formato de resposta esperado do Codex ao concluir uma etapa

```
1. O que foi implementado
2. Arquivos criados/alterados
3. Decisões de arquitetura tomadas
4. Como executar localmente
5. Como testar
6. Resultado dos testes
7. Pendências ou riscos encontrados
8. Confirmacao explícita de que etapas futuras não foram implementadas
```

13. Decisões que devem permanecer configuráveis

Categoria	Exemplos
Modelos	LLM de resposta, LLM interno, embedding model, CrossEncoder reranker.
Retrieval	fetch_k, candidate_k, final_k, lambda_mult, filtros de metadata.
Agentic flow	max_rag_attempts, políticas/thresholds do grader após calibracao.

Web	provider, quantidade de resultados, domínios permitidos/bloqueados, timeout.
Streaming	mensagens de status e limites de payload; contrato de eventos permanece estavel.
Persistência	Chroma local/server/cloud conforme ambiente; checkpointer de dev/produção.
Segurança	auth, rate limits, políticas de acesso e regras de dados por tenant.

14. Evoluções futuras previstas

- Hybrid retrieval: dense + lexical/BM25 + Reciprocal Rank Fusion + CrossEncoder.
- Filtros de metadata por tenant, produto, document type, idioma ou validade.
- Chunking structure-aware/semantic orientado ao corpus real.
- Reranker fine-tuned no domínio quando houver dataset suficiente.
- Cache de embeddings/retrieval e políticas de invalidação.
- Human-in-the-loop para fluxos de alto risco.
- Roteamento adicional para SQL/APIs internas, mantendo o mesmo grafo-orquestrador.
- Avaliações offline e online integradas ao CI/CD.

15. Referências técnicas

As referências abaixo servem para o Codex consultar comportamentos atuais das bibliotecas ao implementar cada etapa. Priorizar documentação oficial em vez de exemplos desatualizados de terceiros.

Referência	URL
LangGraph - Agentic RAG	https://docs.langchain.com/oss/python/langgraph/agentic-rag
LangGraph - Graph API	https://docs.langchain.com/oss/python/langgraph/graph-api
LangGraph - Streaming	https://docs.langchain.com/oss/python/langgraph/streaming
LangGraph - Persistence	https://docs.langchain.com/oss/python/langgraph/persistence
LangChain - Chroma integration	https://docs.langchain.com/oss/python/integrations/vectorstores/chroma
Chroma - Query and Get	https://docs.trychroma.com/docs/querying-collections/query-and-get
Chroma - Metadata filtering	https://docs.trychroma.com/docs/querying-collections/metadata-filtering
Sentence Transformers - Retrieve & Re-Rank	https://www.sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html
Sentence Transformers - CrossEncoder	https://www.sbert.net/docs/cross_encoder/usage/usage.html

16. Comando inicial para o Codex

> **Use este documento como especificação**
>
> Comece somente pela Etapa 1 - Bootstrap da API. Respeite os princípios arquiteturais e os limites de módulo descritos neste PDF. Antes de escrever código, inspecione o repositório existente e adapte a etapa ao padrão já utilizado no projeto. Não implemente Chroma, LangGraph ou qualquer etapa futura neste primeiro passo. Ao concluir, reporte os critérios de aceite e os testes executados.

FIM DA ESPECIFICAÇÃO